

Vue des avions

Javions – étape 9

1. Introduction

Le but de cette étape est d'écrire le code permettant de dessiner les icônes représentant les avions, ainsi que leur trajectoire et une étiquette donnant quelques informations de base à leur sujet.

2. Concepts

2.1. Vue des avions

Nous nommerons **vue des avions** la partie de l'interface graphique qui montre les avions, éventuellement accompagnés de leur trajectoire ou d'une étiquette informative.

L'image ci-dessous montre ce à quoi ressemble cette vue, une fois superposée à la carte de base. On y voit un unique avion, un [avion immatriculé 9H-VCU](#), en phase d'approche de l'aéroport de Genève-Cointrin. L'icône de cet avion donne une idée du type d'appareil dont il s'agit, l'étiquette présente les principales informations à son sujet – immatriculation, vitesse et altitude – et sa trajectoire montre la manœuvre qu'il est en train d'effectuer pour se poser.

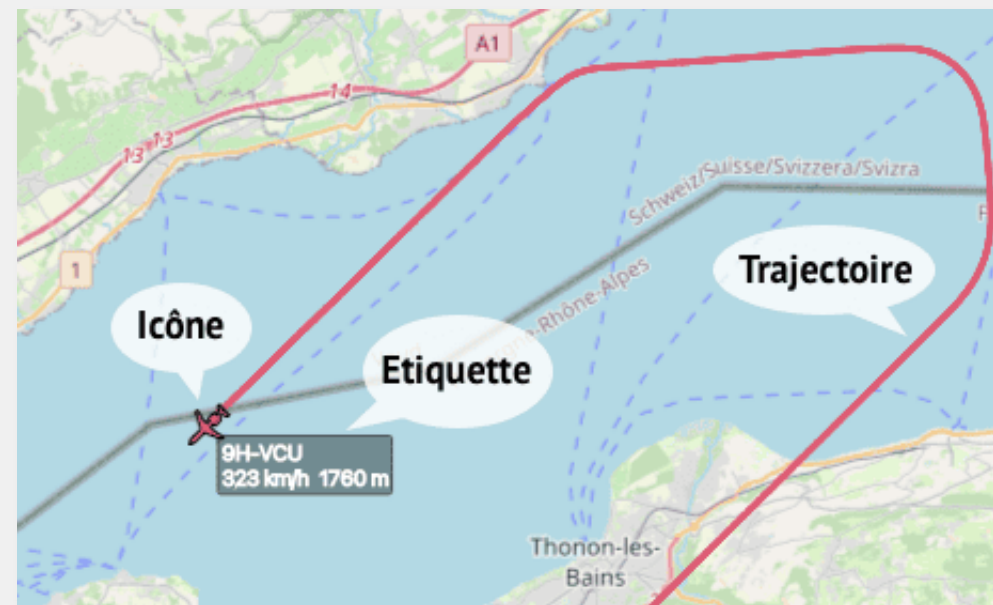


Figure 1 : Vue d'un avion

La couleur de l'icône donne une idée de l'altitude à laquelle se trouve l'avion, les altitudes basses étant colorées avec une couleur plus sombre que les hautes, comme décrit à la section suivante. De même, les segments de droite composant la trajectoire sont colorés chacun au moyen d'un dégradé entre les couleurs correspondant aux

Vue des avions

Javions – étape 9

1. Introduction

Le but de cette étape est d'écrire le code permettant de dessiner les icônes représentant les avions, ainsi que leur trajectoire et une étiquette donnant quelques informations de base à leur sujet.

2. Concepts

2.1. Vue des avions

Nous nommerons **vue des avions** la partie de l'interface graphique qui montre les avions, éventuellement accompagnés de leur trajectoire ou d'une étiquette informative.

L'image ci-dessous montre ce à quoi ressemble cette vue, une fois superposée à la carte de base. On y voit un unique avion, un [avion immatriculé 9H-VCU](#), en phase d'approche de l'aéroport de Genève-Cointrin. L'icône de cet avion donne une idée du type d'appareil dont il s'agit, l'étiquette présente les principales informations à son sujet – immatriculation, vitesse et altitude – et sa trajectoire montre la manœuvre qu'il est en train d'effectuer pour se poser.

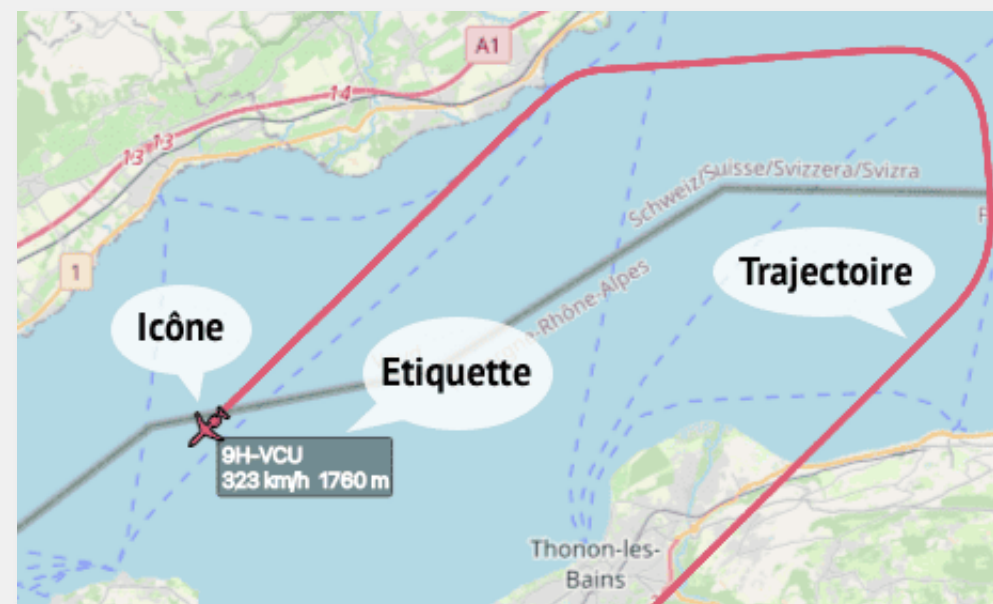


Figure 1 : Vue d'un avion

La couleur de l'icône donne une idée de l'altitude à laquelle se trouve l'avion, les altitudes basses étant colorées avec une couleur plus sombre que les hautes, comme décrit à la section suivante. De même, les segments de droite composant la trajectoire sont colorés chacun au moyen d'un dégradé entre les couleurs correspondant aux

altitudes de leurs extrémités.

Les aéronefs sont superposés en fonction de leur altitude. Ainsi, lorsque les icônes de plusieurs aéronefs se chevauchent, celle correspondant à celui volant le plus haut apparaît devant les autres, comme si les aéronefs étaient vus depuis l'espace.

Afin de ne pas surcharger l'affichage, les étiquettes et les trajectoires ne sont généralement pas visibles. Pour rendre visibles celles d'un aéronef particulier, l'utilisateur a la possibilité de sélectionner celui-ci en cliquant sur son icône – ou en le sélectionnant dans la table comme nous le verrons ultérieurement. De plus, les étiquettes – mais pas les trajectoires – sont toutes visibles lorsque le niveau de zoom est au moins égal à 11.

L'étiquette d'un aéronef affiche, sur deux lignes, les informations suivantes :

- sur la première ligne, son immatriculation si elle est connue, sinon son indicatif s'il est connu, sinon son adresse OACI,
- sur la seconde ligne, sa vitesse en kilomètres par heure et son altitude en mètres ; ces deux valeurs sont suivies chacune du nom abrégé de leur unité (km/h ou m), et séparées l'une de l'autre par [une espace demi-cadratin](#).

Lorsque la vitesse ou l'altitude sont inconnues, un point d'interrogation est affiché à la place de leur valeur, mais l'unité reste. Par exemple, une vitesse inconnue est représentée par la chaîne « ? km/h ».

La trajectoire, l'étiquette et l'icône sont superposées dans cet ordre, c.-à-d. que la trajectoire est placée derrière l'étiquette, elle-même placée derrière l'icône.

2.2. Coloration des altitudes

Les icônes et trajectoires d'aéronefs sont colorés en fonction de l'altitude, au moyen d'un dégradé de couleurs nommé *Plasma*, visible ci-dessous.



Figure 2 : *Plasma* (par Nathaniel J. Smith et Stefan van der Walt)

Ce dégradé peut être vu comme une fonction qui fait correspondre une couleur à un nombre réel : la couleur la plus à gauche – un bleu foncé – correspond aux valeurs inférieures ou égales à 0, tandis que la couleur la plus à droite – un jaune clair – correspond aux valeurs supérieures ou égales à 1. Les couleurs intermédiaires correspondent aux valeurs comprises entre 0 et 1.

Afin d'utiliser ce dégradé pour colorier les icônes et trajectoires en fonction de l'altitude, il faut donc choisir une fonction faisant correspondre la plage dans laquelle les aéronefs volent généralement avec l'intervalle $[0, 1]$. Pour ce projet, nous utiliserons la fonction suivante :

$$c = \left[\frac{a}{12\,000} \right]^{\frac{1}{3}}$$

où a est l'altitude exprimée en mètres, et c est le nombre, compris généralement entre 0 et 1, passé au dégradé afin d'obtenir la couleur correspondante.

La constante 12000 dans la formule ci-dessus correspond approximativement à la plus haute altitude à laquelle volent les avions de ligne, et le rôle de la racine cubique est de distinguer plus finement les altitudes basses, qui sont les plus importantes.

3. Mise en œuvre Java

La mise en œuvre de cette étape n'est pas triviale, et nous vous conseillons donc vivement d'écrire le code en plusieurs phases successives, en le testant petit à petit. Concrètement, nous vous proposons de procéder ainsi :

1. écrire une version de base de la classe `AircraftController` décrite plus bas, qui ne fasse rien d'autre qu'afficher les icônes des aéronefs, dans une couleur unique, sans leur trajectoire et étiquette, et qui ne gère pas les interactions avec l'utilisateur,
2. compléter le programme de test donné à la §3.4 et vérifier que les icônes apparaissent et disparaissent bien, et qu'elles se déplacent et s'orientent correctement,
3. ajouter les étiquettes, en les affichant initialement pour tous les aéronefs, indépendamment du niveau de zoom, et relancer le programme de test pour vérifier leur aspect,
4. procéder de même pour la trajectoire,
5. ajouter la possibilité de sélectionner un aéronef, et gérer correctement l'apparition et la disparition des étiquettes et trajectoires,
6. écrire la classe `ColorRamp` et l'utiliser pour colorier correctement les icônes des aéronefs et leurs trajectoires.

Pour faciliter votre travail, nous vous fournissons [une archive Zip](#) contenant les fichiers suivants, à importer dans votre projet :

- trois fichiers nommés `aircraft.css`, `status.css` et `table.css`, à placer dans le dossier `resources` de votre projet,
- un fichier nommé `AircraftIcon.java`, à placer dans le dossier du sous-paquetage `gui` de votre projet.

Les trois premiers fichiers sont des **feuille de style** (*cascading style sheets* ou CSS en anglais), qui décrivent l'aspect des éléments de l'interface graphique, tandis que le dernier est un fichier Java contenant le type énuméré `AircraftIcon` décrit à la section suivante.

Il n'est pas nécessaire de comprendre le contenu des feuilles de style, qui devront

simplement être attachées à différents nœuds du graphe de scène, comme décrit plus bas. Les personnes intéressées à en savoir plus à leur sujet pourront néanmoins consulter le document [JavaFX CSS Reference Guide](#).

3.1. Type énuméré `AircraftIcon` (fourni)

Le type énuméré `AircraftIcon`, du sous-paquetage `gui`, représente les différentes icônes d'aéronefs disponibles. Les éléments de ce type énuméré possèdent deux méthodes publiques :

- `boolean canRotate()`, qui retourne vrai ssi l'icône peut (et doit) être tournée afin d'indiquer le cap de l'aéronef,
- `String svgPath()`, qui retourne le chemin SVG correspondant à l'icône.

Le chemin SVG est une description du dessin de l'icône, qui peut être placé dans la propriété `content` d'une instance de la classe `SVGPath` pour obtenir un nœud JavaFX représentant l'icône. Ces chemins ont été extraits du fichier `markers.js` de la [version de flightaware du projet dump1090](#).

De plus, la classe `AircraftIcon` possède une méthode publique et statique permettant de déterminer l'icône à utiliser pour un aéronef :

- `AircraftIcon iconFor(AircraftTypeDesignator typeDesignator, AircraftDescription typeDescription, int category, WakeTurbulenceCategory wakeTurbulenceCategory)`, qui retourne l'icône correspondant le mieux à l'aéronef dont les caractéristiques sont celles données – qui ne peuvent pas être nulles.

3.2. Classe `ColorRamp`

La classe `ColorRamp` du sous-paquetage `gui`, publique, finale et immuable, représente un dégradé de couleurs, du type de celui de la figure 2. Comme expliqué plus haut, un tel dégradé est vu comme une fonction associant une couleur à un nombre réel.

Le constructeur de cette classe accepte en argument une séquence de couleurs JavaFX, de type `Color`. Cette séquence est passée sous la forme d'un nombre variable d'arguments – ou éventuellement d'une liste. Si elle ne contient pas au moins deux couleurs, alors le constructeur lève une exception.

Cette séquence de couleurs spécifie la fonction représentée par le dégradé. La première d'entre elles est la couleur correspondant à 0, la dernière à 1, et les autres aux valeurs intermédiaires, réparties régulièrement dans l'intervalle allant de 0 à 1.

Par exemple, si le constructeur est appelé avec 5 couleurs, alors la première correspond à 0, la seconde à 0.25, la troisième à 0.5, la quatrième à 0.75 et la cinquième à 1.

En plus de ce constructeur, la classe `ColorRamp` offre une unique méthode publique,

nommée par exemple `at`, qui prend un argument de type `double` et retourne la couleur correspondante.

Lorsqu'on lui passe une valeur inférieure à 0, elle retourne la première couleur, et lorsqu'on lui passe une valeur supérieure à 1, elle retourne la dernière couleur. Lorsqu'on lui passe une valeur qui se trouve entre deux points pour lesquels une couleur est connue, la couleur retournée est un mélange, dans les bonnes proportions, des couleurs de ces deux points.

Par exemple, si le constructeur a été appelé avec 5 couleurs comme ci-dessus, et que la méthode `at` est ensuite appelée avec 0.3, alors la couleur retournée est un mélange de 80% de la deuxième couleur, et de 20% de la troisième couleur – car 0.3 se trouve à 20% de la distance séparant 0.25 et 0.5.

Le dégradé *Plasma* est défini par la constante suivante, à ajouter à votre classe `ColorRamp`.

```
public static final ColorRamp PLASMA = new ColorRamp(
    Color.valueOf("0x0d0887ff"), Color.valueOf("0x220690ff"),
    Color.valueOf("0x320597ff"), Color.valueOf("0x40049dff"),
    Color.valueOf("0x4e02a2ff"), Color.valueOf("0x5b01a5ff"),
    Color.valueOf("0x6800a8ff"), Color.valueOf("0x7501a8ff"),
    Color.valueOf("0x8104a7ff"), Color.valueOf("0x8d0ba5ff"),
    Color.valueOf("0x9814a0ff"), Color.valueOf("0xa31d9aff"),
    Color.valueOf("0xad2693ff"), Color.valueOf("0xb6308bff"),
    Color.valueOf("0xbf3984ff"), Color.valueOf("0xc7427cff"),
    Color.valueOf("0xcf4c74ff"), Color.valueOf("0xd6556dff"),
    Color.valueOf("0xdd5e66ff"), Color.valueOf("0xe3685fff"),
    Color.valueOf("0xe97258ff"), Color.valueOf("0xee7c51ff"),
    Color.valueOf("0xf3874aff"), Color.valueOf("0xf79243ff"),
    Color.valueOf("0xfa9d3bff"), Color.valueOf("0xfca935ff"),
    Color.valueOf("0xfdb52eff"), Color.valueOf("0xfdc229ff"),
    Color.valueOf("0xfccf25ff"), Color.valueOf("0xf9dd24ff"),
    Color.valueOf("0xf5eb27ff"), Color.valueOf("0xf0f921ff"));
```

3.2.1. Conseils de programmation

La classe `Color` possède une méthode `interpolate` qui permet de mélanger deux couleurs et qui est très utile pour écrire la méthode `at`. Par exemple, utilisée ainsi :

```
Color c1, c2;
Color c = c1.interpolate(c2, 0.2);
```

elle retourne une couleur qui est un mélange de 80% de `c1`, et de 20% de `c2`. Autrement dit, elle retourne une couleur qui se trouve à 20% du dégradé allant de `c1` à `c2`.

3.3. Classe `AircraftController`

La classe `AircraftController` du sous-paquetage `gui`, publique et finale, gère la vue des aéronefs. Son unique constructeur public prend en arguments :

- les paramètres de la portion de la carte visible à l'écran,
- l'ensemble (observable mais non modifiable) des états des aéronefs qui doivent apparaître sur la vue – et qui proviennent bien entendu d'un gestionnaire d'états,
- une propriété JavaFX contenant l'état de l'aéronef sélectionné, dont le contenu peut être nul lorsqu'aucun aéronef n'est sélectionné.

`AircraftController` offre une unique méthode publique, nommée p. ex. `pane`, qui retourne le panneau JavaFX sur lequel les aéronefs sont affichés. Ce panneau est destiné à être superposé à celui montrant le fond de carte.

3.3.1. Graphe de scène

Le graphe de scène de la vue des aéronefs est présentée sur la figure 3 plus bas. Chaque rectangle blanc représente un nœud JavaFX, et certains d'entre eux sont accompagnés d'annotations colorées qui donnent :

- en vert, les feuilles de style à attacher au nœud en plaçant leur nom dans la liste retournée par `getStylesheets`,
- en rouge, l'identité des nœuds, à définir au moyen de la méthode `setId`,
- en bleu, les classes de style à associer au nœud en plaçant leur nom dans la liste retournée par `getStyleClass`.

Comme cette image l'illustre, la vue des aéronefs est une instance de `Pane`, et chaque aéronef est représenté par l'un de ses fils, un groupe dont l'identité est la représentation textuelle de son adresse OACI.

Les deux fils de ce groupe représentent, dans l'ordre, la trajectoire de l'aéronef et un groupe comprenant son étiquette et son icône. La raison pour laquelle l'icône et l'étiquette sont groupées ainsi est que cela facilite leur placement à l'écran.

L'icône est toujours visible, alors que l'étiquette et la trajectoire ne le sont qu'à certaines conditions, comme décrit précédemment.

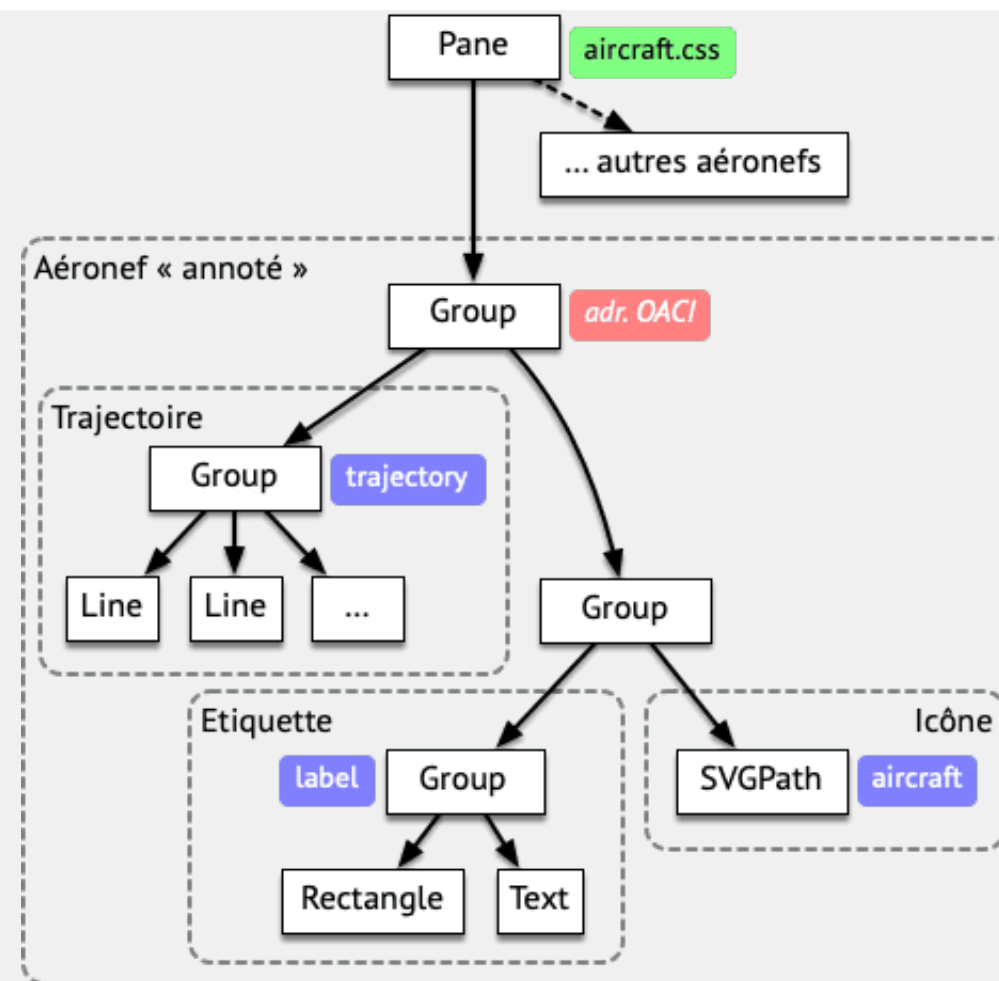


Figure 3 : Graphe de scène de la vue des avions

La vue des avions est destinée à être placée au-dessus de la vue présentant le fond de carte, réalisée à l'étape précédente. Afin que permette à cette dernière de recevoir les événements souris lorsque l'utilisateur clique sur une partie transparente de la vue des avions, il faut impérativement appeler la méthode `setPickOnBounds` du panneau la contenant, en lui passant `false` en argument. Cela peut être fait immédiatement après la création du panneau.

3.3.2. Liens et auditeurs

Un grand nombre de caractéristiques des éléments graphiques présentés à l'écran change au cours du temps. Par exemple, la position de l'icône représentant un avion change lorsque la position de l'avion dans l'espace change – suite à la réception d'un message de position en vol de sa part –, ou lorsque l'utilisateur déplace la carte, ou change son niveau de zoom.

Chaque fois qu'une caractéristique doit ainsi changer au cours du temps, il faut essayer autant que possible d'utiliser les liens (*bindings*) JavaFX pour déterminer sa valeur, car il s'agit de loin de la solution la plus simple et la plus concise. Dans certains cas, cela n'est toutefois pas possible, et il faut alors utiliser directement des auditeurs – c.-à-d. des observateurs.

Les liens à établir ou les auditeurs à installer pour garantir que la vue des avions reste à jour au cours du temps sont décrits rapidement ci-dessous.

1. Ensemble des aéronefs visibles

Les aéronefs visibles sur la vue des aéronefs sont ceux de l'ensemble observable passé au constructeur de `AircraftController`. Cet ensemble doit donc être observé afin que les icônes des aéronefs apparaissent et disparaissent lorsque le contenu de l'ensemble change.

L'observation de l'ensemble est faite par un auditeur, dont l'installation n'est pas totalement triviale et est donc décrite dans les conseils de programmation plus bas.

2. Groupe de l'aéronef annoté

Comme tous les nœuds JavaFX, le groupe contenant l'icône, la trajectoire et l'étiquette possède une propriété nommée `viewOrder` qui détermine l'ordre dans lequel il est dessiné dans son parent – ici la vue des aéronefs.

En liant cette propriété à une expression dont la valeur est égale à la négation de l'altitude de l'aéronef, on garantit que le groupe correspondant à un aéronef volant à une plus haute altitude qu'un autre soit bien dessiné *après* – et donc par dessus – le groupe correspondant à cet autre aéronef.

3. Groupe icône / étiquette

Le groupe contenant l'icône et l'étiquette a pour but de faciliter le placement à l'écran de ces deux éléments, puisque leur position à l'écran doit toujours être égale. Cela peut sembler surprenant, car sur l'image de la figure 1, le coin haut-gauche de l'étiquette est clairement décalé par rapport à l'icône, mais ce décalage est géré par la feuille de style.

Le placement du groupe de l'icône et de l'étiquette se fait au moyen de ses propriétés `layoutX` et `layoutY`. Elles doivent être liées à une expression dépendant de la position de l'aéronef au voisinage de la Terre et des caractéristiques de la portion de la carte visible à l'écran – niveau de zoom et coin haut-gauche.

Par exemple, étant donnée la longitude de la position d'un aéronef, on peut déterminer sa coordonnée x sur la carte du monde au niveau de zoom courant, au moyen des formules données à la §2.5 de l'étape 1 et mises en œuvre dans la classe `WebMercator`. Cela fait, une simple soustraction de la coordonnée x du coin haut-gauche de la portion de la carte visible à l'écran donne la coordonnée x de l'aéronef à l'écran.

4. Icône

L'instance de `SVGPath` représentant l'icône possède un certain nombre de propriétés qui peuvent être utilisées pour garantir qu'elle a la bonne apparence :

- `content`, qui contient le chemin SVG représentant le dessin de l'aéronef, et qui doit être liée à une expression obtenant ce chemin depuis l'instance de `AircraftIcon` correspondant à l'aéronef,
- `rotate`, qui contient l'angle – exprimé en degrés – de la rotation à appliquer à l'icône, et qui doit être liée à une expression égale au cap, en degrés, de l'aéronef si son icône peut être tournée – c.-à-d. si la méthode `canRotate` de l'instance de `AircraftIcon` qui lui correspond retourne vrai –, et à 0 sinon,
- `fill`, qui contient la couleur de remplissage de l'icône, et qui doit être liée à une expression déterminant cette couleur à partir de l'altitude de l'aéronef.

Étant donné que les deux premières propriétés doivent être liées à des expressions qui dépendent de l'instance de `AircraftIcon` correspondant à l'aéronef, il peut être judicieux de définir un lien dont la valeur est justement cette instance.

5. Trajectoire

La trajectoire de l'aéronef est représentée par un groupe de segments de droites reliant les points qui constituent la trajectoire de l'aéronef.

La représentation graphique de la trajectoire doit être recrée chaque fois que la trajectoire elle-même, ou le niveau de zoom de la carte, change. Pour être informé de ces changements, des auditeurs doivent être installés sur les propriétés correspondantes, mais uniquement lorsque la trajectoire est visible, car recréer une trajectoire invisible est inutile et coûteux.

Contrairement à celles de l'icône et de l'étiquette, la position des éléments de la trajectoire ne change pas au cours du temps. En fait, leur position est fixe lorsqu'elle est exprimée dans le système de coordonnées de la carte du monde au niveau de zoom courant. Il est donc conseillé d'exprimer effectivement la position des éléments de la trajectoire dans ce système de coordonnées, puis de lier les propriétés `layoutX` et `layoutY` à des expressions – très simples ! – permettant de les placer correctement à l'écran.

Afin que la trajectoire de l'aéronef ne soit visible que s'il est actuellement sélectionné, sa propriété `visible` doit être liée à une expression dont la valeur n'est vraie que lorsque c'est son propre état qui est contenu dans la propriété passée au constructeur.

6. Étiquette

L'étiquette attachée à l'aéronef est représentée par un texte placé devant un rectangle semi-transparent, qui facilite la lecture.

Le texte est une instance de `Text` dont la propriété `text` doit être liée à une expression produisant la chaîne de l'étiquette.

L'arrière-plan est une instance de `Rectangle`. Sa propriété `width` doit être liée à une expression dont la valeur est égale à la largeur du texte de l'étiquette, plus 4. De même, sa propriété `height` doit être liée à une expression dont la valeur est égale à la hauteur du texte de l'étiquette, plus 4. Les dimensions du texte de l'étiquette s'obtiennent au moyen de la propriété `layoutBounds` de l'instance de `Text`, comme illustrée dans les conseils de programmation plus bas.

Afin que l'étiquette ne soit visible que lorsqu'elle doit l'être, sa propriété `visible` doit être liée à une expression qui n'est vraie que lorsque le niveau de zoom est supérieur ou égal à 11, ou que l'aéronef sélectionné est celui auquel l'étiquette correspond.

3.3.3. Gestion des événements

Le seul événement à traiter est le clic sur l'icône d'un aéronef, qui doit provoquer sa sélection – par modification du contenu de la propriété contenant l'état de l'aéronef sélectionné.

3.3.4. Conseils de programmation

1. Organisation du code

La classe `AircraftController` étant assez longue, il est important de découper son code en plusieurs méthodes privées, chargées chacune de la construction d'une partie du graphe de scène. Par exemple, vous pourriez avoir une méthode nommée `icon` dont le seul but est de construire le graphe de scène correspondant à l'icône de l'aéronef, y compris les liens et les gestionnaires d'événements y relatifs.

2. Observation de l'ensemble des états d'aéronefs

Afin d'observer l'ensemble des états des aéronefs qui doivent apparaître à l'écran, il faut installer un auditeur de type `SetChangeListener<...>` sur l'ensemble passé au constructeur. Cet auditeur reçoit en argument, à chaque changement du contenu de l'ensemble, une valeur de type `Change<...>`. Les méthodes de cette valeur permettent de savoir si le changement est un ajout ou une suppression, et de connaître l'élément ajouté ou supprimé.

L'installation de cet auditeur n'est pas totalement triviale si on utilise une lambda, car la méthode `addListener` est surchargée, et plusieurs de ses variantes acceptent une lambda à un seul argument. Il faut donc utiliser un transtypage explicite afin de forcer le choix de la bonne d'entre elles, ainsi :

```
s.addListener((SetChangeListener<ObservableAircraftState>)
    change -> { /* ... corps de la lambda */ })
```

où `s` est l'ensemble observable contenant les états des aéronefs.

Chaque fois qu'un nouvel aéronef est ajouté à l'ensemble, il faut construire le groupe le représentant – nommé *aéronef* « *annoté* » dans la figure 3 – et l'ajouter aux fils du panneau qui représente la vue des aéronefs. De même, chaque fois qu'un aéronef est supprimé de l'ensemble, il faut supprimer le groupe le représentant des fils du panneau. Le fait que le groupe correspondant à un aéronef ait pour identité la représentation textuelle de son adresse OACI simplifie cette suppression.

3. Valeurs changeant au cours du temps

Comme nous l'avons vu, un très grand nombre de caractéristiques des nœuds JavaFX apparaissant à l'écran doit changer au cours du temps, en fonction, entre autres, des changements de l'état des aéronefs.

La manière la plus simple d'effectuer ces changements est d'utiliser le système de liens de JavaFX. Il est donc capital de bien le comprendre avant de se lancer dans l'écriture du code.

Il est en particulier important de comprendre que, si ce mécanisme est basé sur le patron *Observer*, dans la plupart des cas il n'est pas nécessaire de travailler directement avec des observateurs – nommés auditeurs dans JavaFX –, car il est possible de travailler à un plus haut niveau d'abstraction.

Par exemple, pour que les aéronefs soient dessinés dans le bon ordre, il faut, comme nous l'avons vu, faire en sorte que la valeur de la propriété `viewOrder` du groupe représente un aéronef soit toujours égale à la négation de son altitude. Cela peut se faire très simplement en liant cette propriété à une expression obtenue au moyen de la méthode `negate` :

```
ObservableAircraftState s = /* état de l'aéronef */;  
Group g = /* groupe de l'aéronef annoté */;  
g.viewOrderProperty().bind(s.altitudeProperty().negate());
```

Même si en pratique `negate` utilise un observateur pour être informée des changements de la propriété dont elle doit produire la négation – ici l'altitude de l'aéronef –, cet observateur n'apparaît pas dans le code ci-dessus.

De manière similaire, la méthode `map` permet d'appliquer une transformation quelconque, spécifiée généralement au moyen d'une lambda, à une valeur changeant au cours du temps. Ainsi, pour s'assurer que la largeur du rectangle d'arrière-plan de l'étiquette d'un aéronef est toujours égale à la largeur de l'étiquette plus 4, on peut écrire :

```
Text t = /* texte de l'étiquette */;  
Rectangle r = /* rectangle d'arrière-plan */;  
r.widthProperty().bind(  
    t.layoutBoundsProperty().map(b -> b.getWidth() + 4));
```

Lorsqu'on travaille avec des chaînes de caractères, la méthode `format` de `Bindings` peut être très utile. Elle est similaire à la méthode du même nom de la classe `String`, et permet donc de formater des chaînes de caractères au moyen d'expressions de formatage du type de celles utilisées par `printf`. La différence est que les arguments qu'on lui passe peuvent être des valeurs qui changent au cours du temps, et que son résultat est aussi une chaîne qui change au cours du temps.

Par exemple, pour faire en sorte que le texte d'une instance de `Text` donne toujours l'altitude en mètres d'un aéronef, on peut écrire :

```
Text t = ...;
ObservableAircraftState s = ...;
t.textProperty().bind(
    Bindings.format("%f mètres", s.altitudeProperty()));
```

Finalement, dans certains cas, aucune des méthodes mentionnées plus haut ne convient. On peut alors souvent utiliser une des méthodes `create...Binding` de `Bindings`, qui permettent de créer un « lien » dont la valeur est déterminé par un morceau de code quelconque, donné généralement par une lambda.

Par exemple, le code ci-dessus pourrait être récrit au moyen de la méthode `createStringBinding` ainsi :

```
Text t = ...;
ObservableAircraftState s = ...;
t.textProperty().bind(
    Bindings.createStringBinding(() ->
        String.format("%f mètres", s.getAltitude()),
        s.altitudeProperty()));
```

Le premier argument passé aux méthodes `create...Binding` est une lambda sans argument qui produit une valeur du type désiré, ici une chaîne. Cette lambda utilise généralement une ou plusieurs valeurs qui varient au cours du temps, ici l'altitude de l'aéronef. Ces valeurs sont ce que l'on nomme les dépendances du lien, et elles doivent être toutes passées en arguments, après la lambda.

Comme cet exemple l'illustre, les méthodes `create...Binding` sont moins agréables à utiliser que les méthodes spécialisées décrites précédemment, mais elles sont également plus puissantes, et donc indispensables dans certains cas.

4. Positionnement de l'icône et de l'étiquette

La position sur la carte du groupe constitué de l'icône et de l'étiquette de l'aéronef peut se déterminer en projetant, au moyen des méthodes de `WebMercator`, sa position dans l'espace. Ensuite, une simple soustraction – qui effectue le

changement entre le système de coordonnées de la carte du monde entier et celui de la vue – permet d'en calculer sa position dans la vue des aéronefs.

Le calcul de cette position est une des situations dans laquelle l'utilisation d'une méthode `create...Binding` – ici `createDoubleBinding` – est judicieuse.

5. Dessin de l'étiquette

Le point de code Unicode de l'espace demi-cadratin est 2002_{16} , donc une manière de l'insérer dans une chaîne est d'utiliser la syntaxe d'échappement avec `\u`, par exemple `"une\u2002espace"`.

6. Coloration de la trajectoire

Comme dit plus haut, la trajectoire est représentée par un groupe de segments de droites, chacun d'entre eux étant une instance de `Line`. La « couleur » utilisée pour dessiner chacun de ces segments, passée à `setStroke`, est soit :

- une couleur simple, de type `Color`, lorsque l'altitude du segment est constante,
- un dégradé, de type `LinearGradient`, sinon.

La documentation de `LinearGradient` n'étant pas très claire, voici un extrait de code montrant comment construire un dégradé entre les couleurs `c1` et `c2` :

```
Stop s1 = new Stop(0, c1);
Stop s2 = new Stop(1, c2);
new LinearGradient(0, 0, 1, 0, true, NO_CYCLE, s1, s2);
```

3.4. Tests

Pour tester cette étape, vous pouvez vous inspirer de l'application ci-dessous, qui construit une interface graphique très simple, constituée d'une carte de base au-dessus de laquelle volent les aéronefs. Les deux sont superposés dans une instance de `StackPane`, un panneau qui empile ses fils les uns au-dessus des autres.

La méthode `readAllMessages` lit la totalité des messages du fichier binaire fourni, et les retourne dans une liste. Son code, laissé en exercice, est très similaire à celui de l'exemple de l'étape 7.

Afin que les aéronefs se déplacent, un « minuteur d'application » – une sous-classe anonyme de `AnimationTimer` – est créé puis démarré au moyen de sa méthode `start`. Le code de sa redéfinition de la méthode `handle`, que JavaFX appelle environ 60 fois par seconde, prend les 10 prochains messages et les fournit au gestionnaire d'état des aéronefs (`AircraftStateManager`). Notez bien que ce code a de nombreux problèmes : d'une part il ne vérifie pas qu'il reste bien des messages, donc plante dès que

l'itérateur n'en a plus à fournir, et d'autre part il ne respecte absolument pas l'horodatage des messages. Toutefois, comme son but est uniquement de vérifier que les aéronefs sont correctement dessinés, il suffit largement.

```
public final class TestAircraftController extends Application{
    public static void main(String[] args) { launch(args); }

    static List<RawMessage> readAllMessages(String fileName)
        throws IOException { /* ... à faire */ }

    @Override
    public void start(Stage primaryStage) throws Exception {
        // ... à compléter (voir TestBaseMapController)
        BaseMapController bmc = ...;

        // Création de la base de données
        URL dbUrl = getClass().getResource("/aircraft.zip");
        assert dbUrl != null;
        String f = Path.of(dbUrl.toURI()).toString();
        var db = new AircraftDatabase(f);

        AircraftStateManager asm = new AircraftStateManager(db);
        ObjectProperty<ObservableAircraftState> sap =
            new SimpleObjectProperty<>();
        AircraftController ac =
            new AircraftController(mp, asm.states(), sap);
        var root = new StackPane(bmc.pane(), ac.pane());
        primaryStage.setScene(new Scene(root));
        primaryStage.show();

        var mi = readAllMessages("messages_20230318_0915.bin")
            .iterator();

        // Animation des aéronefs
        new AnimationTimer() {
            @Override
            public void handle(long now) {
                try {
                    for (int i = 0; i < 10; i += 1) {
                        Message m = MessageParser.parse(mi.next());
                        if (m != null) asm.updateWithMessage(m);
                    }
                } catch (IOException e) {
                    throw new UncheckedIOException(e);
                }
            }
        };
    }
}
```

```
    }  
    }.start();  
  }  
}
```

La vidéo ci-dessous montre comment ce programme de test devrait se comporter une fois l'étape terminée.



4. Résumé

Pour cette étape, vous devez :

- écrire les classes `ColorRamp` et `AircraftController` selon les indications données ci-dessus,
- tester votre code,
- documenter la totalité des entités publiques que vous avez définies.

Aucun rendu n'est à faire pour cette étape avant le rendu final. N'oubliez pas de faire régulièrement des copies de sauvegarde de votre travail en suivant [nos indications à ce sujet](#).